



Proof-of-Compute for Federated Language Model Training: A Cryptographic Framework

Antje Worring & Zach Kelling

Zoo Labs Foundation

January 2025

Abstract

We present a **Proof-of-Compute** (PoC) framework for verifying that a participant in decentralized language model training actually performed the claimed computation. PoC combines three mechanisms: (1) **Ed25519 delta signatures**—each training delta is signed by the worker’s key, binding the worker identity to the computation output; (2) **gradient hashing**—intermediate gradient states are committed via SHA3-256 hash chains, creating a tamper-evident log of the training trajectory; (3) **spot-verify sampling**—the orchestrator probabilistically re-executes a subset of training steps and compares results within a tolerance bound. We prove that a rational adversary cannot profitably fake compute under the PoC framework when the spot-check probability exceeds $p > c_{\text{fake}} / (c_{\text{fake}} + s \cdot f_{\text{slash}})$, where c_{fake} is the cost of faking, s is the stake, and f_{slash} is the slash fraction. For Zoo Gym’s parameters ($p = 0.1$, $s = 10,000$ GYM, $f_{\text{slash}} = 0.5$), faking is unprofitable unless the adversary can produce indistinguishable fake deltas at less than 2% of real training cost. We also describe slashing conditions, the hash-chain audit trail, and the path toward zero-knowledge proofs of training for full verification without re-execution.

Keywords: proof-of-compute, Ed25519, gradient hashing, spot-check verification, slashing, zero-knowledge proofs

1 Introduction

In centralized training, the training operator trusts that its own GPUs performed the computation. In decentralized training (Zoo Gym), nodes are untrusted: a rational node may claim rewards for compute it never performed. Attacks include:

1. **Free-riding:** Submitting zero or random deltas to collect participation rewards.
2. **Replay:** Resubmitting deltas from a previous epoch.
3. **Proxy:** Outsourcing compute to a cheaper (but lower-quality) provider and pocketing the margin.
4. **Gradient theft:** Stealing another node’s delta and claiming it as one’s own.

We need a verification system that makes these attacks economically unprofitable without requiring every delta to be fully re-verified (which would negate the throughput benefit of decentralization).

1.1 Contributions

1. **Three-layer PoC framework:** Signatures bind identity, hash chains bind sequence, spot-checks bind correctness.
2. **Economic security model:** Formal conditions under which faking is unprofitable, parameterized by spot-check rate and stake/slash ratio.
3. **Gradient hash chain:** Incremental commitment scheme that allows efficient audit of the training trajectory.
4. **Spot-verify protocol:** Randomized re-execution with deterministic seeding to ensure reproducibility.
5. **Path to zkPoC:** Architecture for zero-knowledge proofs of training steps, providing full verification without re-execution.

2 Threat Model

2.1 Adversary Capabilities

The adversary controls up to $f < N/3$ nodes and can:

1. Produce arbitrary delta payloads.
2. Observe all public communications (delta hashes, ring messages).
3. Collude across controlled nodes.
4. Invest up to budget B in computational resources.

The adversary *cannot*:

1. Forge Ed25519 signatures (computational hardness of DLP on Curve25519).
2. Predict the spot-check selection (random oracle model).
3. Corrupt the blockchain state (honest majority assumption on Hanzo chain).

2.2 Security Goals

Definition 2.1 (Compute Authenticity). *A delta δ is authentic if it was produced by executing the specified training step on the specified data with the specified model weights.*

Definition 2.2 (Non-Repudiation). *A node that submits delta δ with signature σ cannot later deny having produced δ .*

Definition 2.3 (Freshness). *A delta δ for epoch e , step t cannot be a replay of a delta from any other $(e', t') \neq (e, t)$.*

3 Ed25519 Delta Signatures

3.1 Key Registration

Each node generates an Ed25519 keypair (sk, pk) at registration. The public key pk is recorded on-chain in the orchestrator registry contract.

```
1 use ed25519_dalek::{SigningKey, VerifyingKey};
2 use rand::rngs::OsRng;
3
4 pub fn register_node() -> (SigningKey, VerifyingKey) {
5     let sk = SigningKey::generate(&mut OsRng);
6     let pk = sk.verifying_key();
7     // Record pk on-chain
8     chain.register_key(node_id, pk.as_bytes());
9     (sk, pk)
10 }
```

Listing 1: Key generation and registration

3.2 Delta Signing

Every delta includes a signature over the tuple $(e, t, H(\delta))$:

$$\sigma = \text{ED25519.SIGN}(sk, e\|t\|\text{SHA3-256}(\delta)) \quad (1)$$

where e is the epoch, t is the step, and δ is the serialized delta payload (header + tensors, per the Delta binary format).

Theorem 3.1 (Unforgeability). *Under the hardness of the Discrete Logarithm Problem on Curve25519, no PPT adversary can produce a valid (δ', σ') for a node it does not control, with probability greater than 2^{-128} .*

This follows directly from the EUF-CMA security of Ed25519 [1].

3.3 Freshness

Including (e, t) in the signed message prevents replay: a signature valid for (e_1, t_1) is invalid for (e_2, t_2) .

Proposition 3.2 (Anti-Replay). *A delta signed for (e, t) cannot be replayed at (e', t') without forging a new signature.*

3.4 Non-Repudiation

The on-chain binding of pk to `node_id` means that a valid signature constitutes proof of authorship. The node cannot deny having produced the delta.

4 Gradient Hash Chain

4.1 Construction

At each training step t , the node commits the intermediate gradient state to a hash chain:

$$h_t = \text{SHA3-256}(h_{t-1}\|t\|\text{SHA3-256}(\nabla_t)) \quad (2)$$

with $h_0 = \text{SHA3-256}(\text{"GENESIS"}\|e\|\text{node_id})$.

```
1 import hashlib
2
3 class GradientHashChain:
4     def __init__(self, epoch: int, node_id: str):
5         genesis = f"GENESIS|{epoch}|{node_id}".encode()
6         self.current = hashlib.sha3_256(genesis).digest()
7         self.chain = [self.current]
8
9     def commit(self, step: int, gradient_bytes: bytes):
10        grad_hash = hashlib.sha3_256(gradient_bytes).digest()
11        payload = (
12            self.current
13            + step.to_bytes(8, "little")
14            + grad_hash
15        )
16        self.current = hashlib.sha3_256(payload).digest()
```

```

17         self.chain.append(self.current)
18     return self.current
19
20     def verify(self, step: int, gradient_bytes: bytes,
21               expected_hash: bytes) -> bool:
22         grad_hash = hashlib.sha3_256(gradient_bytes).digest()
23         prev = self.chain[step]
24         payload = (
25             prev
26             + step.to_bytes(8, "little")
27             + grad_hash
28         )
29         computed = hashlib.sha3_256(payload).digest()
30     return computed == expected_hash

```

Listing 2: Gradient hash chain (Python)

4.2 Properties

Theorem 4.1 (Tamper Evidence). *If an adversary modifies any gradient ∇_t after committing h_t , all subsequent hashes $h_{t+1}, h_{t+2}, \dots$ become invalid. Specifically, for the chain to remain consistent, the adversary must find a preimage collision in SHA3-256, which has complexity $O(2^{128})$.*

Proof. The hash chain is a Merkle list. Modifying ∇_t produces ∇'_t with $\text{SHA3-256}(\nabla'_t) \neq \text{SHA3-256}(\nabla_t)$ (collision resistance). This yields $h'_t \neq h_t$, which propagates to $h'_{t+1} \neq h_{t+1}$, etc. The final hash h_T no longer matches the committed value. $\square$

Proposition 4.2 (Efficient Audit). *Given the full chain $\{h_0, \dots, h_T\}$ and any single gradient ∇_t , verification of step t requires $O(1)$ hash computations.*

4.3 Submission

At epoch end, the node submits:

- The final hash h_T (32 bytes).
- The delta δ (compressed tensors).
- The Ed25519 signature σ over $(e, T, h_T, \text{SHA3-256}(\delta))$.

The orchestrator stores h_T for potential spot-check verification.

5 Spot-Verify Sampling

5.1 Protocol

5.2 Deterministic Seeding

For spot-checks to be reproducible, the training step must be deterministic given:

1. Model weights θ_{t-1} (from checkpoint).

Algorithm 1 Spot-Verify Protocol

Require: Delta δ , hash chain tip h_T , spot-check probability p

```
1: Draw  $u \sim \text{Uniform}(0, 1)$ 
2: if  $u < p$  then
3:   // Select random step to verify
4:    $t^* \leftarrow \text{Uniform}(\{1, \dots, T\})$ 
5:   Request gradient  $\nabla_{t^*}$  and chain prefix  $\{h_0, \dots, h_{t^*}\}$  from node
6:   // Re-execute step  $t^*$ 
7:   Load model state at step  $t^* - 1$  from checkpoint
8:   Load batch  $B_{t^*}$  (deterministic from step number + dataset seed)
9:    $\nabla'_{t^*} \leftarrow \text{FORWARD} + \text{BACKWARD}(\theta_{t^*-1}, B_{t^*})$ 
10:  // Verify
11:  if  $\|\nabla'_{t^*} - \nabla_{t^*}\|_2 / \|\nabla'_{t^*}\|_2 > \epsilon$  then
12:    Flag node for slashing
13:  end if
14:  // Verify hash chain consistency
15:  Recompute  $h_{t^*}$  from  $h_{t^*-1}, t^*, \nabla_{t^*}$ 
16:  if  $h_{t^*}$  does not match submitted chain then
17:    Flag node for slashing
18:  end if
19: end if
```

2. Batch B_t (determined by $\text{seed} = \text{SHA3-256}(e\|t\|\text{dataset_cid})$).
3. Random number generator state (seeded by step number).

```
1 import hashlib, torch
2
3 def get_batch_indices(epoch: int, step: int,
4                       dataset_cid: str,
5                       dataset_size: int,
6                       batch_size: int) -> list[int]:
7     seed_bytes = hashlib.sha3_256(
8         f"{epoch}|{step}|{dataset_cid}".encode()
9     ).digest()
10    seed = int.from_bytes(seed_bytes[:8], "little")
11    rng = torch.Generator().manual_seed(seed)
12    indices = torch.randperm(
13        dataset_size, generator=rng
14    )[:batch_size]
15    return indices.tolist()
```

Listing 3: Deterministic batch selection

5.3 Tolerance Bound

Due to floating-point non-determinism across hardware, we allow a relative tolerance:

$$\epsilon = \frac{\|\nabla' - \nabla\|_2}{\|\nabla'\|_2} \leq 0.05 \quad (3)$$

This accommodates differences in GPU precision (FP16 rounding, fused multiply-add order) while detecting gross discrepancies (random gradients, zero gradients, stale replays).

Lemma 5.1 (Tolerance Soundness). *For honest computation, the relative error between two executions of the same step on different hardware satisfies $\epsilon_{\text{honest}} < 0.01$ with probability > 0.999 . The threshold $\epsilon = 0.05$ provides a $5\times$ margin.*

This was measured empirically across A100, RTX 4090, and RTX 4080 hardware.

6 Economic Security

6.1 Adversary’s Decision

A rational adversary chooses between:

1. **Honest:** Pay cost c_{real} per step, earn reward r .
2. **Fake:** Pay cost $c_{\text{fake}} < c_{\text{real}}$ per step (e.g., submitting random data), earn reward r if undetected, lose stake $s \cdot f_{\text{slash}}$ if detected.

6.2 Expected Payoff

$$\mathbb{E}[\pi_{\text{honest}}] = r - c_{\text{real}} \quad (4)$$

$$\mathbb{E}[\pi_{\text{fake}}] = (1 - p_d) \cdot r - c_{\text{fake}} - p_d \cdot s \cdot f_{\text{slash}} \quad (5)$$

where p_d is the detection probability. From the quality scoring, a fake delta gets $q_i \approx 0$ (zero convergence contribution), so in practice $r_{\text{fake}} \approx 0$ even if undetected. This strengthens the bound, but we analyze the worst case where $r_{\text{fake}} = r$.

6.3 Incentive Compatibility Condition

Honest computation dominates when:

$$\mathbb{E}[\pi_{\text{honest}}] > \mathbb{E}[\pi_{\text{fake}}] \quad (6)$$

Substituting and simplifying:

$$p_d \cdot (r + s \cdot f_{\text{slash}}) > c_{\text{real}} - c_{\text{fake}} \quad (7)$$

Theorem 6.1 (Minimum Spot-Check Rate). *Faking is unprofitable when the spot-check probability satisfies:*

$$p > \frac{c_{\text{real}} - c_{\text{fake}}}{r + s \cdot f_{\text{slash}}} \quad (8)$$

Parameter	Value
Spot-check probability p	0.10
Minimum stake s	10,000 GYM
Slash fraction f_{slash}	0.50
Reward per epoch r	8.4 GYM
Real compute cost c_{real}	\$2.50 (GPU-hour)

Table 1: Zoo Gym economic parameters.

Category	Detection	Slash	Evidence
Invalid signature	Signature check	5%	(δ, σ, pk)
Hash chain break	Chain verification	10%	(h_t, h_{t+1}, ∇_t)
Spot-check failure	Re-execution	25%	$(\nabla, \nabla', \epsilon)$
Gradient poisoning	Statistical outlier	50%	Norm/direction analysis
Collusion	Cross-node correlation	50%	Correlated deltas

Table 2: Slashing categories, detection methods, and penalties.

6.4 Zoo Gym Parameters

For faking to be profitable:

$$c_{\text{fake}} < c_{\text{real}} - p \cdot (r + s \cdot f_{\text{slash}}) \quad (9)$$

$$c_{\text{fake}} < 2.50 - 0.10 \cdot (8.4 + 5,000) = 2.50 - 500.84 < 0 \quad (10)$$

Since $c_{\text{fake}} \geq 0$, the condition is *never satisfiable*: faking is always unprofitable under Zoo Gym’s parameters. The massive stake-to-reward ratio ($5,000 : 8.4 \approx 595 : 1$) provides overwhelming security margin.

Corollary 6.2. *Even with spot-check probability $p = 0.001$ (0.1%), faking remains unprofitable as long as $s \cdot f_{\text{slash}} > (c_{\text{real}} - c_{\text{fake}})/p$. For $c_{\text{real}} = \$2.50$ and $c_{\text{fake}} = \$0$: $s \cdot f_{\text{slash}} > \$2,500$, which holds for a 10,000 GYM stake valued at $> \$5$ per GYM.*

7 Slashing Conditions

7.1 Offense Categories

7.2 Evidence Format

Slashing requires on-chain evidence submission:

```

1 contract SlashingJury {
2   struct Evidence {
3     address accused;
4     uint8 category; // 0-4 per table
5     bytes32 deltaHash;
6     bytes proof;
7     uint256 timestamp;

```



```

8      }
9
10     function submitEvidence(Evidence calldata ev)
11         external onlyOrchestrator {
12             require(verifyProof(ev), "Invalid proof");
13             uint256 slashPct = slashRates[ev.category];
14             gymToken.slash(
15                 ev.accused, slashPct, msg.sender
16             );
17             emit NodeSlashed(
18                 ev.accused, ev.category, slashPct
19             );
20         }
21
22     function verifyProof(Evidence calldata ev)
23         internal view returns (bool) {
24         if (ev.category == 0) {
25             // Invalid signature: verify that sig
26             // does NOT match registered key
27             return !ed25519Verify(
28                 ev.deltaHash,
29                 ev.proof,
30                 registeredKeys[ev.accused]
31             );
32         } else if (ev.category == 1) {
33             // Hash chain break: verify
34             // recomputed hash differs
35             return verifyHashChainBreak(ev.proof);
36         }
37         // ... other categories
38         return false;
39     }
40 }

```

Listing 4: Slashing evidence contract

7.3 Progressive Penalties

First offense: base rate (per Table 2). Repeat offenses within 30 days: $2\times$ base rate. Three offenses in 30 days: full stake slash and permanent ban.

7.4 Appeal Process

Slashed nodes may appeal within 7 days by submitting counter-evidence. A governance vote (simple majority of GYM stakers) can reverse the slashing if the evidence is found insufficient.

8 Toward Zero-Knowledge Proofs of Training

The spot-check approach requires the orchestrator to re-execute training steps, which is computationally expensive. Zero-knowledge proofs of training (zkPoC) would allow full verification without re-execution.

8.1 Architecture

A zkPoC system proves the statement:

“I executed forward pass f and backward pass b on batch B with weights θ , producing gradient ∇ , such that $\|\nabla - \nabla_{\text{claimed}}\|_2 / \|\nabla\|_2 < \epsilon$.”

8.2 Challenges

1. **Circuit size:** A single forward pass of a 7B parameter model involves $\sim 10^{12}$ floating-point operations. Current zkSNARK circuits cannot handle this directly.
2. **Floating-point arithmetic:** ZK circuits operate on finite fields, not IEEE 754 floats. Fixed-point approximation introduces errors.
3. **Proving time:** Even with recursive SNARKs, proving a single training step may take hours.

8.3 Tractable Approach: Partial Verification

Instead of proving the entire training step, prove a subset:

1. **Commitment:** Prove that $H(\nabla) = h_{\text{claimed}}$ (the gradient hash matches the chain entry). This is already done by the hash chain.
2. **Structural validity:** Prove that ∇ has the correct shape and dtype for the model architecture. This is a simple constraint circuit.
3. **Statistical properties:** Prove that $\|\nabla\|_2 \in [0.5\bar{n}, 2\bar{n}]$ (gradient norm within expected range). This requires $O(|\theta|)$ multiplications.
4. **Random projection:** Prove that $\langle \nabla, r \rangle = v$ for a random vector r chosen by the verifier. By the Johnson-Lindenstrauss lemma, this preserves distance with high probability.

Proposition 8.1 (Random Projection Verification). *If a prover claims gradient ∇ and the verifier checks k random projections $\langle \nabla, r_j \rangle = v_j$, a fake gradient ∇' that differs from ∇ by $\geq \delta$ in any coordinate is detected with probability $\geq 1 - (1 - \delta^2 / \|\nabla\|^2)^k$. For $k = 128$ and $\delta / \|\nabla\| = 0.01$, detection probability exceeds $1 - 10^{-5}$.*

```
1 pub fn generate_projection_proof(  
2     gradient: &Tensor,  
3     verifier_seed: &[u8; 32],  
4     num_projections: usize,  
5 ) -> Vec<(Vec<u8>, f64)> {  
6     let mut proofs = Vec::new();  
7     for j in 0..num_projections {  
8         let r = deterministic_random_vector(  
9             verifier_seed, j, gradient.len()  
10        );  
11        let projection = gradient.dot(&r);  
12        let r_hash = sha3_256(&r.to_bytes());  
13        proofs.push((r_hash.to_vec(), projection));  
14    }  
15    proofs
```

```

16 }
17
18 pub fn verify_projection_proof(
19     claimed_gradient: &Tensor,
20     proofs: &[(Vec<u8>, f64)],
21     verifier_seed: &[u8; 32],
22     tolerance: f64,
23 ) -> bool {
24     for (j, (r_hash, claimed_proj)) in
25         proofs.iter().enumerate() {
26         let r = deterministic_random_vector(
27             verifier_seed, j, claimed_gradient.len()
28         );
29         assert_eq!(&sha3_256(&r.to_bytes())[..], &r_hash[..]);
30         let actual_proj = claimed_gradient.dot(&r);
31         if (actual_proj - claimed_proj).abs()
32             / actual_proj.abs() > tolerance {
33             return false;
34         }
35     }
36     true
37 }

```

Listing 5: Random projection proof generation

8.4 zkML Research Directions

Full zkPoC for training steps requires advances in:

1. **zkML frameworks:** EZKL [8], zkLLM [9], and Modulus Labs’ ZKML work toward proving inference. Training proofs are harder (backward pass + optimizer state).
2. **Recursive SNARKs:** Nova [10] and Supernova enable incremental verification, amortizing proof cost across steps.
3. **Hardware acceleration:** FPGA-based provers [11] can parallelize MSM computations required for large circuits.

9 Evaluation

9.1 Setup

32-node ring training Qwen3-2507 (QLoRA, 50 epochs, 100K samples). Injected attacks: 3 nodes submitting random gradients, 2 nodes replaying stale deltas, 1 node submitting poisoned gradients (biased toward a specific output).

9.2 Detection Results

Random gradients are detected immediately by the quality scoring system ($q_i = 0$ due to zero convergence contribution). Replay attacks are detected by the epoch/step binding in Ed25519 signatures. Gradient poisoning (subtle bias) requires spot-check re-execution and is detected within 4 epochs.

Attack	Nodes	Detected	Epochs to Detect
Random gradients	3	3/3	1 (quality score = 0)
Replay (stale delta)	2	2/2	1 (signature check)
Gradient poisoning	1	1/1	4 (spot-check)

Table 3: Attack detection results.

Spot-Check Rate	Overhead	Detection Rate (poison)
5%	2.1% wall-clock	41% after 10 epochs
10%	4.3% wall-clock	65% after 10 epochs
20%	8.7% wall-clock	88% after 10 epochs

Table 4: Spot-check overhead and detection effectiveness.

9.3 Spot-Check Overhead

At 10% spot-check rate, the wall-clock overhead is 4.3%—the orchestrator must re-execute 10% of steps, but this is parallelized with ongoing training.

9.4 Hash Chain Verification

9.5 Random Projection Proofs

Random projection proofs are fast enough for per-step verification. 128 projections detect any gradient deviation $>1\%$ of norm with probability > 0.99999 .

10 Related Work

Proof-of-useful-work. Primecoin [4] proves prime finding; Gridcoin [5] credits BOINC compute. Neither provides verifiable ML training proofs.

Verifiable computation. Pinocchio [6] and Groth16 [7] prove arbitrary circuits but are infeasible for large ML models.

zkML. EZKL [8] proves neural network inference in ZK circuits. zkLLM [9] extends this to transformer attention. Neither addresses training verification.

Trusted execution. Intel SGX [12] and ARM TrustZone provide hardware attestation but require specific hardware and have been subject to side-channel attacks [13].

Optimistic verification. Gensyn [14] uses probabilistic spot-checking similar to our approach but lacks the economic security formalization and gradient hash chain.

Federated learning verification. Bonawitz et al. [15] study secure aggregation in federated learning. Our work focuses on verifying individual contributions rather than aggregate privacy.

11 Conclusion

The Proof-of-Compute framework provides layered security for decentralized training: Ed25519 signatures for identity binding, SHA3-256 hash chains for trajectory integrity, and spot-verify sampling for correctness. Under Zoo Gym’s parameters, faking compute is always unprofitable—the stake-to-reward ratio provides a 595 : 1 security margin. Random projection proofs offer a practical

Metric	Value
Hash chain entries per epoch	2,000
Chain verification time (single step)	0.4 ms
Full chain verification (per epoch)	0.8 s
Storage per node per epoch	64 KB

Table 5: Hash chain performance metrics.

Projections (k)	Proof Size	Gen. Time	Verify Time
32	1.2 KB	12 ms	8 ms
64	2.4 KB	24 ms	16 ms
128	4.8 KB	48 ms	31 ms

Table 6: Random projection proof performance (LoRA delta, 16M parameters).

stepping stone toward full zkPoC, enabling per-step verification at 48ms per proof. The path to zero-knowledge proofs of training is open but requires advances in zkML circuit design and recursive SNARK efficiency.

Acknowledgments

This work was supported by Zoo Labs Foundation.

References

- [1] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B. Yang, “High-Speed High-Security Signatures,” *J. Cryptographic Engineering*, 2(2):77–89, 2012.
- [2] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning,” *arXiv:2402.03300*, 2024.
- [3] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [4] S. King, “Primecoin: Cryptocurrency with Prime Number Proof-of-Work,” 2013.
- [5] Gridcoin, “Rewarding BOINC Contributions with Blockchain,” Whitepaper, 2014.
- [6] B. Parno, J. Howell, C. Gentry, and M. Raykova, “Pinocchio: Nearly Practical Verifiable Computation,” *IEEE S&P*, 2013.
- [7] J. Groth, “On the Size of Pairing-Based Non-Interactive Arguments,” *EUROCRYPT*, 2016.
- [8] EZKL, “EZKL: Easy Zero-Knowledge Inference,” <https://github.com/zkonduit/ezkl>, 2023.
- [9] H. Sun, J. Hulse, and T. Ristenpart, “zkLLM: Zero Knowledge Proofs for Large Language Models,” *arXiv:2404.16109*, 2024.
- [10] A. Kothapalli, S. Setty, and I. Tzialla, “Nova: Recursive Zero-Knowledge Arguments from Folding Schemes,” *CRYPTO*, 2022.

- [11] Y. Zhang et al., “PipeZK: Accelerating Zero-Knowledge Proof with a Pipelined Architecture,” *ISCA*, 2021.
- [12] V. Costan and S. Devadas, “Intel SGX Explained,” *Cryptology ePrint Archive*, 2016.
- [13] J. Van Bulck et al., “Foreshadow: Extracting the Keys to the Intel SGX Kingdom,” *USENIX Security*, 2018.
- [14] Gensyn, “Gensyn Protocol: Trustless Training of Deep Neural Networks,” Whitepaper, 2023.
- [15] K. Bonawitz et al., “Towards Federated Learning at Scale: A System Design,” *SysML*, 2019.
- [16] D. Yin et al., “Byzantine-Robust Distributed Learning,” *ICML*, 2018.
- [17] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks,” *AISTATS*, 2017.
- [18] P. Blanchard et al., “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent,” *NeurIPS*, 2017.
- [19] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *ICLR*, 2017.
- [20] Opentensor Foundation, “Bittensor: A Peer-to-Peer Intelligence Market,” Whitepaper, 2023.
- [21] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [22] T. Detrmers et al., “QLoRA: Efficient Finetuning of Quantized Language Models,” *NeurIPS*, 2023.
- [23] H. Touvron et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [24] J. Benet, “IPFS – Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014.
- [25] V. Buterin, “Ethereum Whitepaper,” 2014.
- [26] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning,” *arXiv:1802.05799*, 2018.
- [27] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2022.
- [28] P. Patarasuk and X. Yuan, “Bandwidth Optimal All-Reduce Algorithms,” *J. Parallel Distrib. Comput.*, 2009.
- [29] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Whitepaper, 2017.
- [30] NIST, “Post-Quantum Cryptography Standards: ML-KEM, ML-DSA, SLH-DSA,” FIPS 203/204/205, 2024.